DAILYOFFICE · PLAYBOOK

Product Directory — User Guide

Vendor products → the right Shopify websites, priced, written and published automatically.

For the owner and staff · updated November 2026 · also in the app: Playbook → Guide

Contents

01 What the Product Directory does

02 Before you start: categories, the AI reader and vendors

03 Step 1 — Add and connect each Shopify store

04 Step 2 — Decide which products go to this website

05 Step 3 — Set the selling price

06 Step 4 — Tags in this website's structure

07 Step 5 — Titles, descriptions and SEO

08 Step 6 — Publishing mode, the Active switch and pausing

09 Connect Telegram — vendor posts become products by themselves

10 Daily use — adding a product

11 What each status means

12 Working with listings

13 Troubleshooting

14 Questions people ask

01 What the Product Directory does

One place for every product your vendors send — and a robot that puts each one on the right Shopify websites, at the right price, in the right words.

Every product a vendor sends lands in **Products**. From there the app works on its own, every minute: it reads the photos and the vendor's text, decides which of your Shopify websites the product belongs on, works out the selling price for each website, writes a title, description and Google (SEO) text in that website's own style, adds that website's tags, and puts the product on the store.

How one product travels

1. **Lands** — a product comes in. It comes straight from the vendor's Telegram channel (see **Connect Telegram**), or you add it in **Products → + Add product**. Status: **New**.
2. **Read** — the AI looks at the photos and the vendor's post and fills in name, category, fabric, colour, sizes, cost price and other details (work, occasion, wash care...). Status: **Reading**, then **Ready** — or **Needs info** when the cost price or the category is missing.
3. **Sorted** — each website's **Routing rules** decide whether the product goes there. Every "yes" becomes a *listing*: one product on one website.
4. **Priced and written** — for each listing the price is worked out from the cost with that website's formula, and that website's AI writer writes the title, description, SEO text, tags and photo descriptions.
5. **Published** — the listing is sent to Shopify: live, as a draft, or held until someone approves it — whatever that website's **Publishing mode** says.

> **Tip** One product can be on several websites, each with its own price, title, description and tags. Changing one website never changes another.

Where things are

PAGE	WHAT IT IS FOR
Dashboard	What needs a person today, products by status, and each website's numbers (in progress, needs review, published, failed). Tap any tile to see those products.
Products	Every product. Add new ones, search and filter, open one to see its facts and its listing on each website.
Websites	Your Shopify stores. Each has six tabs: **Connection**, **Publishing**, **Routing rules**, **Pricing**, **Tags**, **Content & AI**.
Vendors	The people you buy from — name, a short code and their Telegram channel.
Settings	The AI that reads products, the category list, **Telegram** (bot, admin group, crawler), and the switch that pauses all automation (plus **Run now**).
Guide	This guide, with **Download PDF**.

Who can do what

- Admins, managers and operators can use every page: add and edit products, approve, publish, change website settings.
- Only an **admin or manager** can save a store's credentials (the Shopify keys), the Telegram bot token and the crawler key.

02 Before you start: categories, the AI reader and vendors

Ten minutes, once. Everything after this depends on it.

Categories

1. Open **Settings → Categories**.
2. Type each category you sell and press Enter — for example `Saree`, `Kurta Set`, `Lehenga`, `Dupatta`, `Co-ord Set`. You can paste several at once, separated by commas.
3. Tap **Save categories**.

The AI must pick one of these for every product. A product without a category waits in **Needs info**. Routing rules, tags and the Shopify product type all use the category, so spell each one exactly the way you want it to appear on your websites.

The AI that reads products

1. In **Settings → AI that reads products**, pick a **Model** and an **Effort**. `Haiku 5.5` with `Model default` is enough for reading.
2. Tap **Save reader**.

This AI only reads (fills in the facts). Each website chooses its own writer later, under **Websites → Content & AI**.

Vendors

1. Open **Vendors → + Add vendor**.
2. **Name**: the vendor as you know them, e.g. `Rangoli Creations`.
3. **Short code** (optional): two or three letters, e.g. `RG`. Tags can use it as `{vendor_code}`.
4. Tap **Save vendor**. Repeat for each vendor.

> **Tip** Stopped buying from someone? Tap **Deactivate** instead of deleting. Their old products stay; they just stop showing up as a choice for new ones.

03 Step 1 — Add and connect each Shopify store

Do this once for every website. You need the store's myshopify.com address and a Shopify login

that is allowed to install apps (the store owner, or staff with that permission).

Add the website

1. Open **Websites → + Add website**.
2. **Name**: how you call it, e.g. `Indya Ethnic`.
3. **Store domain**: the myshopify.com address, e.g. `indya-ethnic.myshopify.com` — find it in Shopify admin → Settings → Domains. Not your own .in / .com domain.
4. Tap **Add website**. The website opens on its **Connection** tab.

A new website starts **Off** with the routing set to **Do not list it**, so nothing is sent until you have finished setting it up.

Which way to connect?

YOUR SITUATION	WHAT YOU USE	IN THE APP PICK
The store already has an older custom app made inside the Shopify admin, with an Admin API access token that starts `shpat_`	That app's access token	**Access token**
Anything else — a new store, or no such app	A new app from the Shopify Dev Dashboard	**Client ID + secret**

> **Note** Shopify stopped letting anyone create new "legacy" custom apps in the store admin on 1 January 2026. Old ones keep working. Every new connection is made in the Shopify Dev Dashboard (Way B).

Way A — an existing custom app (token starting shpat_)

1. In the Shopify admin open Settings → Apps and sales channels → Develop apps, and open the app.
2. Check its Admin API scopes include `write_products`, `read_products`, `write_publications`, `read_publications` — plus `write_inventory` if this website will track stock. If you add any, save and reinstall the app when Shopify asks.
3. Copy the **Admin API access token** (it starts with `shpat_`). Shopify shows it only once — if nobody saved it, use Way B instead.
4. In the app: **Websites →** your website **→ Connection**. Under **How the app signs in** pick **Access token**, paste it into **Admin API access token**, tap **Save credentials**.

Way B — a new app from the Shopify Dev Dashboard

1. Sign in to the Shopify Dev Dashboard (dev.shopify.com) with a Shopify account in the **same organisation as the store**. An app from another organisation cannot connect to the store.
2. Create an app, e.g. `DailyOffice Product Directory`.

3. In the app's access settings add the scopes `write_products`, `read_products`, `write_publications`, `read_publications` — plus `write_inventory` if this website will track stock. Release that version.
4. Install the app on the store and approve it.
5. Open the app's settings and copy the **Client ID** and the **Client secret**.
6. In the app: **Websites →** your website **→ Connection**. Under **How the app signs in** pick **Client ID + secret**, paste both, tap **Save credentials**.

> **Tip** Shopify renames its screens now and then. If a label above looks different, follow the same idea: create app → give permissions (scopes) → release → install on the store → copy client ID and secret.

Test, then switch on

1. Tap **Test connection**.
2. Green **Connected to <store name>. All needed permissions are there.** — done.
3. Yellow **Missing permissions: …** — add those scopes to the app in Shopify, reinstall / approve it, then **Test connection** again.
4. Red **Last problem: …** — the details are wrong or the app was removed. Copy them again, **Save credentials**, test again.
5. Finish Steps 2 to 6 below for this website. Then come back to **Connection** and switch **Active** on.

- **Active** can only be switched on after a successful connection. Switching it on also sends the Ready products already in the directory that match this website's rules. Off = nothing new is sent to this store; products already there stay.
- Credentials are kept on the server only and are never shown again — the boxes stay empty. Leave them empty to keep what is saved; type only to replace.
- With **Client ID + secret** the app renews its Shopify access by itself. Nothing to do.
- Moving a website to a different Shopify store: change **Store domain**, enter that store's credentials and tap **Save credentials**. The website is switched off, the old store's keys are deleted, and its products are queued to be sent again to the new store. Products already on the old store stay there.

04 Step 2 — Decide which products go to this website

Websites → your website → Routing rules.

When no rule matches

CHOICE	WHAT IT MEANS
Do not list it (safe default)	Nothing goes to this store unless a rule sends it. Recommended.

CHOICE	WHAT IT MEANS
List everything	Every product goes to this store unless a rule keeps it off.

This choice is saved with **Save changes** at the bottom of the screen. Rules save on their own.

Add a rule

1. Tap **+ Add rule**.
2. **Rule name** — for you, e.g. `Silk sarees from Rangoli` or `Nothing under ₹500`.
3. **When a product matches** — **Send it to this store** or **Keep it off this store**.
4. Fill only the conditions you need. Every condition you fill must match; empty ones are ignored: **Vendors**, **Categories**, **Fabrics** (any of these, capital letters do not matter), **Cost from ₹** / **Cost up to ₹** (the vendor cost), **Keywords** (any of these words in the name, SKU, details or what the AI read).
5. Optional, for "Send" rules: switch on **Own price for these products** and set a **Markup %** and **Add ₹** that replace the website's own for these products.
6. Tap **Save rule**.

Order matters

Rules are checked from the top. The **first** switched-on rule that matches decides, and the rules below it are not looked at. Use the ↑ ↓ buttons to change the order. Put "Keep it off" rules above wide "Send" rules.

#	RULE	SHOWN AS
1	Nothing under ₹300	**KEEP OFF** · cost ₹0–₹300
2	Bridal lehengas	**SEND** · Lehenga · words: bridal · own price: +150% + ₹200
3	Silk sarees from Rangoli	**SEND** · vendor Rangoli · Saree · fabric Silk

With "When no rule matches" set to **Do not list it**: a ₹250 silk saree from Rangoli is stopped by rule 1. A ₹1,800 silk saree from Rangoli is sent by rule 3. A cotton kurta matches no rule, so it is not listed.

- A rule with no conditions matches every product (the app warns you).
- The switch on a rule turns it off without deleting it.
- A change applies to products that arrive or are checked again after it. For the products already in the directory, tap **Send matching products now** under the rules (or open one product and tap **Check websites again**).
- Deleting a rule never removes products that are already listed.

05 Step 3 — Set the selling price

Websites → your website → Pricing. The price in the vendor's post is your COST; each website turns it into its own selling price.

Selling price = cost × (1 + Markup % ÷ 100) + Add ₹, then rounded, and never below the minimum price.

BOX	WHAT TO TYPE
Markup %	`100` doubles the cost, `80` adds 80%.
Add ₹ (after markup)	A fixed amount on top — shipping, packing, payment charges.
Rounding	How the final number looks (table below).
Compare-at % (was price)	`0` = no struck-out price. `30` = show a "was" price 30% higher.
Minimum price ₹	Never sell below this. Empty = no minimum.

Worked example: cost ₹640, Markup 80%, Add ₹60

640 × 1.8 = 1,152, plus 60 = **₹1,212** before rounding.

ROUNDING	HOW IT WORKS	₹1,212 BECOMES
No rounding	Kept as it is (to the paisa)	₹1,212
Nearest 10	To the nearest ten	₹1,210
Ends in 99	Up to the next ...99. A price already ending in 99 stays (₹1,299 stays; ₹1,300 → ₹1,399)	₹1,299
Ends in 49 / 99	Up to the next ...49 or ...99	₹1,249

Compare-at (the struck-out "was" price)

With **Ends in 99** and **Compare-at %** `30`: 1,299 × 1.3 = 1,688.70, rounded the same way → shoppers see ₹1,699 struck out next to **₹1,299**.

Minimum price

Cost ₹250: 250 × 1.8 + 60 = 510 → Ends in 99 → ₹599. With **Minimum price ₹** `799` the website sells it at **₹799**. The minimum is applied after rounding, so choose a "nice" number like 799 or 999.

A rule's own price

The "Bridal lehengas" rule above uses +150% + ₹200. Cost ₹6,000: 6,000 × 2.5 + 200 = 15,200 → Ends in 99 → **₹15,299**. Rounding, compare-at and the minimum still come from the Pricing tab.

> **Tip** Use the **Preview** box: type a **Sample cost ₹** to see the price, the struck-out price and your margin before you tap **Save changes**.

New pricing applies to listings written from now on. A listing already on Shopify keeps its price until its product is checked again: open the product and tap **Check websites again** — the new price goes to Shopify and the text stays. A price typed by hand (**kept**) never changes on its own.

06 Step 4 — Tags in this website's structure

Websites → your website → Tags. Each store filters and builds collections from tags in its own way — one may use "Fabric_Silk", another plain "silk". This tab sets the pattern for this store.

- **Tag templates** — each line becomes one tag. Type it and press Enter. Words in curly brackets are filled from the product.
- **Fixed tags** — added to every product sent to this store, e.g. `New Arrival`.
- **Letter case** — **As written**, **lowercase** or **Title Case**.
- **AI extra tags** — switch on to let the AI add style / occasion tags it sees in the photos and text, up to **At most** tags.

PLACEHOLDER	FILLED WITH	EXAMPLE
`{category}`	The category	Saree
`{fabric}`	The fabric	Banarasi Silk
`{color}`	The colour	Maroon
`{vendor}`	Vendor name	Rangoli
`{vendor_code}`	Vendor short code	RG
`{size}`	One tag per size	Size_S, Size_M, Size_L
`{sku}`	Product code	RG-101
`{attr:occasion}`	Any detail the AI read — work, occasion, pattern, set_contents, wash_care…	Festive

Example

Templates `{category}`, `Fabric_{fabric}`, `Color_{color}`, `Size_{size}`, `Vendor_{vendor_code}`, `Occasion_{attr:occasion}` and the fixed tag `New Arrival`, for a maroon Banarasi silk saree from Rangoli (RG), Free Size, festive, give:

`Saree` `Fabric_Banarasi Silk` `Color_Maroon` `Size_Free Size` `Vendor_RG` `Occasion_Festive` `New Arrival`

- If a placeholder is empty for a product, that tag is skipped — you never get a half tag like "Fabric_".
- Commas inside a tag become spaces (Shopify splits tags at commas). Repeats are removed, whatever the capital letters.
- The **Example** box at the bottom of the tab shows the tags for a sample product as you type.
- Shopify's product type is set to the category automatically.

07 Step 5 — Titles, descriptions and SEO

Websites → your website → Content & AI. For every listing one AI call writes the title, description, SEO title, SEO description, web address (handle), extra tags and photo descriptions — in this website's voice.

Model and effort

MODEL	GOOD FOR	COST
Haiku 5.5	Fast; good for most listings	Lowest
Sonnet 5.5	Better writing — your main store	About 20 × Haiku
Opus 5.5	The best writing — a premium store	Highest

Effort (**Model default**, **Low**, **Medium**, **High**, **Extra high**, **Max**) is how hard the model thinks before writing. Higher is slower and costs more. **Model default** suits most listings; models that do not support effort ignore it.

> **Tip** Start every website on Haiku 5.5 with Model default. Read 10–20 listings. Move a website up to Sonnet only if its writing is not good enough.

The other settings

SETTING	WHAT IT DOES
Language	`English`, `Hinglish`, `Hindi`...
Brand voice	How this store talks, e.g. "premium, elegant, festive — for brides and wedding guests".
Image alt text	On: the AI describes each photo for Google and for screen readers.
Title pattern	`{ai_title}` lets the AI write the title. Or build it: `{color} {fabric} {category}` → "Maroon Banarasi Silk Saree". Also usable: `{vendor}`, `{sku}`, `{name}`; mix them, e.g. `{ai_title} {sku}`.
Length limits	**Title (characters)**, **Description (words)**, **SEO title (characters)** (Google shows about 60), **SEO description (characters)** (about 155).
Extra instructions	Anything to always do or avoid, e.g. "Mention free shipping above ₹1,999. Never use the word cheap."

> **Note** The AI is told to use only the facts given and never to invent a fabric or size. Still, read the first listings of every new website before you set it to Live.

Descriptions use only simple formatting (paragraphs, bullet points, bold, italics, line breaks), so they look right on any Shopify theme. Save the tab with **Save changes**.

08 Step 6 — Publishing mode, the Active switch and pausing

Websites → your website → Publishing.

PUBLISHING MODE	WHAT HAPPENS	USE IT WHEN
Live	Published and visible on the store straight away	You trust this website's setup
Draft	Sent to Shopify as a draft — someone makes it visible inside Shopify	You want a last look inside Shopify
Review first	Waits in Products as **Needs review** until someone taps **Approve**, then publishes	A new website, the first weeks

Track stock — on: Shopify counts stock and stops selling at 0 (the app needs `write_inventory`). New products start at 0, so they cannot be bought until you set their quantities in Shopify. Off: always orderable — right for made-to-order or vendor-held stock.

Three ways to stop things

SWITCH	WHERE	WHAT IT DOES
Active (one website)	Websites → website → Connection	Off = nothing new is sent to this store. Products already there stay. On again = the matching products are sent.
Rule switch (one rule)	Websites → website → Routing rules	That rule stops sorting products.
Automation (everything)	Settings → Automation	Paused = nothing is read, written or published on its own. Buttons like **Publish now** still work. The Dashboard shows a yellow banner.

Settings → Run now does one round straight away (it can take up to 2 minutes) and shows how many products were read, routed, written, published and failed.

A safe first day for a new website

1. Set **Publishing mode** to **Review first** and tap **Save changes**.
2. On **Connection**, switch **Active** on.
3. Add three or four real products in **Products**.

4. Open each one, open this website's listing, read it, fix anything, tap **Approve**.
5. Check the products on the store. Happy? Switch **Publishing mode** to **Live**.

09 Connect Telegram — vendor posts become products by themselves

Do this once. After it, every product a vendor posts in their Telegram channel lands in Products within a few minutes, with the vendor already filled in — taken from the channel, because the post itself never names the vendor.

How it works

1. A **crawler** — one of your staff Telegram accounts, running on a small cloud server all day — reads the vendor channels you list in Playbook.
2. It **forwards** each new post into your private **admin group**.
3. The **Playbook bot** is an admin in that group. It passes every forwarded message to Playbook.
4. Playbook joins the photos, videos and text of one post into one product and sets the vendor from the channel. Then the usual steps run: read by the AI, sorted, priced, written, published.

You need: a Telegram account to make the bot, a staff Telegram account for the crawler (a normal account, not a bot — bots cannot read other people's channels), and a small cloud server (Railway, about US$5 a month).

1 · Make the Playbook bot

1. In Telegram open **@BotFather** and send `/newbot`.
2. Give it a name (e.g. `DailyOffice Playbook`) and a username that ends in `bot` (e.g. `dailyoffice_playbook_bot`).
3. BotFather sends a **token** (numbers, a colon, then letters). Copy it.
4. In Playbook open **Settings → Telegram → 1 · Playbook bot**, paste it into **Bot token** and tap **Save token**.
5. You see the bot's @name and **Connected**. If it says **Telegram can't reach Playbook**, tap **Check bot**; if that does not clear it, save the token again.

- Only an **admin or manager** can save the token. Everyone else sees the status.
- The token is kept on the server only and is never shown again.

2 · Link your admin group

1. In Telegram make a **private group** (e.g. `Vendor posts`) with you and the **staff account** that will run the crawler.
2. Add the Playbook bot to the group and make it an **admin**.

3. In Playbook: **Settings → Telegram → 2 · Admin group → Get link code** (admins and managers only). A 6-digit code appears, valid for 30 minutes; 5 wrong tries in any group and it stops working.
4. Send `/link 123456` in the group (your code instead of 123456). Or tap **Copy /link …** in Playbook and paste it.
5. The bot answers **Linked to Playbook** ✓ and Settings shows **Linked** with the group's name — no reload needed.

> **Tip** Keep the group just for this. Messages typed there by people are ignored; only forwarded posts become products.

- Then lock it down: in **@BotFather** send `/setjoingroups`, pick the bot and choose **Disable**, so nobody can add it to other groups (turn it back on for a moment if you ever link another group).
- In the staff account: **Settings → Privacy and Security → Groups & Channels → My Contacts**, so strangers cannot pull it into their groups.

3 · Start the crawler

1. On a laptop open **my.telegram.org**, sign in with the **staff account's** phone number, open **API development tools**, create an app (any name) and copy **api_id** and **api_hash**.
2. In Playbook: **Settings → Telegram → 3 · Crawler → Create crawler key → Copy key**. It is shown only once.
3. On the laptop, in the project folder `tools/pb-telegram-crawler`, run `npm ci` then `npm run login`. Type the api_id, api_hash, the staff phone number, the code Telegram sends and the two-step password if there is one. It prints **TG_SESSION** once.
4. On **railway.com**: New Project → Deploy from GitHub repo → allow access to **kreeva-lifestyle / unsort** → pick it. In the service settings set **Root Directory** to `tools/pb-telegram-crawler`, **Restart Policy** to **Always** and **Region** to Singapore.
5. Under **Variables** add `TG_API_ID`, `TG_API_HASH`, `TG_SESSION`, `PB_CRAWLER_KEY` (the key from Playbook), `PB_URL` = `https://ulphprdnswznfztawbvg.supabase.co/functions/v1/pb-telegram` and `RAILWAY_DEPLOYMENT_OVERLAP_SECONDS` = `0`. Deploy.
6. Within a minute **Settings → Telegram → 3 · Crawler** shows **Running**, the version, and each vendor channel with its last post number.

> **Note** **TG_SESSION is a login.** Anyone who has it can read and send messages as the staff account. Paste it only into the server's variables — never into a chat, email or WhatsApp. To cancel it: Telegram on the staff phone → Settings → Devices → end that session, then run `npm run login` again.

- Any Linux server with Docker works instead of Railway — the steps are in the crawler's README.
- Run only **one** crawler. Two copies at once can make Telegram end the staff account's session, and would forward every post twice.
- The first time it sees a channel it starts after the newest post, so old posts are not copied.

- **Replace crawler key** makes a new key and stops the old one at once — paste the new one into `PB_CRAWLER_KEY` on the server straight away.

Then: give each vendor their channel

1. Open **Vendors → Edit** on a vendor.
2. In **Telegram channel** type the channel name — `@kumari_art`, `kumari_art` or a link like `t.me/kumari_art` all work. Tap **Save vendor**.
3. From the staff account, **join** the channel (needed for private channels; good practice for all).

The vendor row then shows the channel, e.g. `@kumari_art`. A post from a channel that no vendor has yet still comes in: Playbook makes a new vendor named after the channel. Rename it under **Vendors** if you like. When a vendor reposts another channel's post, it still goes to the vendor whose channel it was copied from.

What happens to a vendor post

IN THE CHANNEL	IN PLAYBOOK
Albums of photos (up to 10 each)	Up to 20 photos on one product, in the order posted
A video up to 20 MB	Copied next to the photos (with a play button). Videos never go to Shopify or the AI
A bigger video	Skipped, with a note on the product: "Video (91 MB) too big to copy — add it by hand"
The long text (CODE, SIZE, COLOUR, RATE...), or a photo caption	The product's **Details** — the AI reads it
A short message like `200 Less` after the rate (also `Less: 200`, `Less Rs 200`, or `RATE 1600 LESS 200` on one line)	Kept as the **Less**: cost = rate − less (₹1,600 − ₹200 = ₹1,400), shown in **Facts**. A new product waits 10 minutes after its text before the AI reads it, so a Less sent a little later still counts
A pointing-hand sticker or emoji between products	Treated as a separator and ignored
Photos with no text after an hour	A product with a note "No description arrived from Telegram". If the text comes later (within a day), it is added and the product is read again
The text first, then its photos	Also one product — each text goes with the photos after it
The same post copied twice	Counted once

A product from Telegram shows **From Telegram · <channel> · Open post** under its name, and anything to check in a yellow **Notes from Telegram** tile. **Messages waiting** in Settings counts forwarded messages not yet turned into products — it goes back to 0 a few minutes after a post ends.

When something is wrong

SETTINGS SHOWS	WHAT TO DO
Telegram can't reach Playbook on the bot	Tap **Check bot**. Still red: **Change token** and save the same token again
Waiting for /link does not change	The bot must be an **admin** in the group, and the code must be under 30 minutes old — tap **New code** and send it again. The bot says "Too many wrong codes" after 5 wrong tries: get a new code
Crawler **Stopped**	Open the server (Railway → Deploy Logs) and restart it. A login error means TG_SESSION was ended: run `npm run login` again
"Join this channel from the staff account"	Join the vendor's channel from the staff account
"This channel blocks forwarding"	The vendor turned on "Restrict saving content". Ask them to turn it off, or add those products by hand
"Could not forward — check the staff account is still in the admin group"	Add the staff account back to the group. Nothing is skipped meanwhile
"Telegram asked us to slow down ..."	Telegram's speed limit. Nothing to do — it carries on by itself
"... being tried again" next to Messages waiting	Nothing to do — Playbook tries those messages again by itself (up to 3 times)
"... could not be used" next to Messages waiting	They failed three times. Open **See which posts**, tap **Open post** and add that product by hand; tell the developer if it keeps happening

10 Daily use — adding a product

1. Tap **+ Add product** (on **Products** or on the **Dashboard**).
2. **Photos** — add one or more (JPG, PNG or WEBP). The first photo is the main one.
3. **Vendor** — pick one, or create a new vendor right there.
4. **Cost price (₹)** — the vendor's price to you.
5. **SKU** — optional, your product code.
6. **Details** — paste the vendor's whole post: fabric, colour, sizes, work, price.
7. **Category** and **Sizes** — optional. Leave **Let the AI decide** and the AI fills them in.
8. Tap **Add product**. The product opens; within a minute the AI reads it and the websites pick it up.

- You need at least a photo, the details text or a SKU.
- Whatever you type yourself is kept exactly as you wrote it — the AI only fills empty boxes.

> **Tip** The better the post you paste, the better the listing. Fabric, colour and sizes in the text help the AI more than the photo alone.

11 What each status means

Product status

STATUS	WHAT IT MEANS	WHAT YOU DO
New	Waiting for the AI to read it	Nothing — within a minute
Reading	The AI is reading it now	Wait; editing is off for a moment
Needs info	Cost price or category is missing — or the AI could not read it after 3 tries (red "AI reader" note)	Open it, fill **Cost price (₹)** / **Category** in **Facts**, tap **Save facts** — it becomes Ready and is sent to the websites. Both already filled? Tap **Mark ready**
Ready	Read, and checked against every website's rules	Nothing. See where it went in the **Websites** box
Archived	Hidden from the list and not sent anywhere	**Restore** brings it back

Listing status (one product on one website)

YOU SEE	INTERNAL NAME	WHAT IT MEANS	WHAT YOU DO
Queued	`queued`	Waiting to be written	Nothing
Writing	`generating`	The AI is writing title, description, SEO and tags	Wait; editing is off
Needs review	`review`	Written; this website is on **Review first**	Open, check, edit, **Approve**
Publishing	`publishing`	Being sent to Shopify	Wait; editing is off
Published	`published`	On Shopify	**Shopify** opens it in the Shopify admin
Failed	`failed`	Shopify or the AI refused; the red text says why	See "Fixing a failed listing"
Removed	`skipped`	Taken off this website (Draft on Shopify)	**Put back on site** if needed

In the **Products** status filter the listing statuses start with "Listing:". **Listing: In progress** means Queued, Writing and Publishing together.

12 Working with listings

Open a product, then tap a website in its **Websites** box. The listing opens with its price, title, description, SEO, handle and tags, and the buttons below.

Fixing a failed listing

1. On the **Dashboard** tap **Failed to publish** (or filter Products by **Listing: Failed**).
2. Open the product, then the website shown with the red text.
3. Read the red text and fix the cause — see **Troubleshooting**.
4. Tap **Publish now** (or **Update on Shopify** if it is already there).

The app also tries again by itself — up to 3 times, 10 minutes apart — so a short Shopify hiccup clears on its own.

Editing, and "kept" fields

Change any of **Price (₹)**, **Title**, **Description**, **SEO title**, **SEO description**, **Handle (URL)** and **Tags (comma list)**, then tap **Save changes**. Each field you change gets a small **kept** label: **Regenerate** will never touch it. Product **Facts** work the same way — a box you typed is kept and the AI reader never overwrites it.

> **Tip** To hand a field back to the AI: empty the box, tap **Save changes**, then **Regenerate**. The **kept** label goes, and from then on the AI (and, for the price, the cost and the website's pricing) decides that field again. This works for price, title, description and both SEO fields.

- On a published listing, after **Save changes** tap **Update on Shopify** to send the change.
- While a listing is being written or sent to Shopify, editing is off. Close it and open it again in a minute.
- Once a product is on Shopify its web address (handle) never changes, so old links keep working.

The buttons

BUTTON	WHAT IT DOES
Regenerate	Writes the listing again with the website's current settings and works the price out again. Kept fields stay. On a Live or Draft website it is then sent to Shopify by itself within a minute; on Review first it waits for Approve.
Approve	Review first websites: sends the listing to Shopify within a minute. While **Automation** is paused it waits — tap **Publish now** to send it. A website that is switched off cannot approve.
Publish now / **Update on Shopify**	Sends it to Shopify straight away instead of waiting.
Remove from site	Sets the product to Draft on Shopify — hidden from shoppers, not deleted. Status **Removed**. It is not re-listed on its own.

BUTTON	WHAT IT DOES
Put back on site	Sends a removed listing to Shopify again. One removed before the AI had written it is written first, then sent.
Open in Shopify	Opens the product in that store's Shopify admin.

On the product itself

BUTTON	WHAT IT DOES
Read again with AI	The AI reads the photos and text again and refreshes every field it filled. Kept fields stay.
Check websites again	Runs every website's rules again for this product — use it after you change a rule. Published listings also get the price from today's cost and pricing, sent to Shopify without rewriting the text (a **kept** price stays).
Save facts	Saves your changes. If the cost, category, vendor, fabric, name, SKU or details changed, the websites are checked again: unpublished listings are re-priced and rewritten, published ones are re-priced and updated on Shopify (their text stays; a **kept** price stays).
Archive / **Restore**	Archive hides the product and stops sending it anywhere. Listings already on Shopify stay — use **Remove from site** first if they should go. Restore makes the AI read it again.
+ Add photos	Adds more photos to the product.

13 Troubleshooting

WHAT YOU SEE	LIKELY REASON	WHAT TO DO
Test connection fails, or a red "Last problem"	Wrong or old token / secret, the app was uninstalled, or the store domain is not the myshopify.com one	Copy the details again from Shopify, **Save credentials**, **Test connection**
"Missing permissions: …"	The app in Shopify does not have every scope	Add the listed scopes to the app, release / reinstall it, test again
Client ID + secret refused although they are right	The app is in a different Shopify organisation, or not installed on this store	Create the app in the store's own organisation and install it on the store
Active will not switch on	The store has never passed a test	**Save credentials**, then a green **Test connection**
Products stay **New** for more than a few minutes	Automation is paused, or a big batch is being worked through	Dashboard banner → **Open Settings** → switch Automation on; or tap **Run now**

WHAT YOU SEE	LIKELY REASON	WHAT TO DO
Product in **Needs info**	Cost price or category missing, or the AI could not read it (red "AI reader" note)	Fill the facts → **Save facts** (already filled: **Mark ready**); or **Read again with AI**
Product **Ready** but on no website	No rule matched and the website is on **Do not list it**; or the website is Off	Check the rules (conditions, order, switches) and **Active**, then **Check websites again**
Product went to the wrong website	A wide rule matched first	Move a "Keep it off" rule above it; **Remove from site** on the wrong one
Failed: "The AI answer was cut off — try a lower effort setting"	Effort too high for the length limits	**Content & AI** → lower **Effort** → **Save changes** → **Regenerate**
Published, but shoppers cannot see it	The website is on **Draft**; or the publications permission is missing (noted in the listing's red text)	Make it visible in Shopify, or add `read_publications` + `write_publications` and tap **Update on Shopify**
The price is not what you expected	Markup, rounding, minimum or a rule's own price — or the price is **kept**	Try the cost in the **Pricing → Preview** box; check the rule; empty the price box → **Save changes** → **Regenerate**
A tag is missing	Its placeholder is empty for this product (e.g. no fabric)	Fill that fact on the product → **Save facts** → **Regenerate**
Listing boxes are greyed out	It is being written or sent to Shopify right now	Close it and open it again in a minute
"Product saved, but the photos did not upload"	Weak internet, or the file type	On the product tap **+ Add photos** (JPG, PNG or WEBP)

14 Questions people ask

Does the app ever delete a product from Shopify?

No. **Remove from site** only sets it to Draft. **Archive** does not touch Shopify at all.

I changed a website's pricing / tags / writing. Are old products updated?

Not by themselves. New listings use the new settings. For a new **price** on a product already on Shopify, open the product and tap **Check websites again** — the price is updated on Shopify and the text stays. For new tags or writing, open the listing and tap **Regenerate** (and **Update on Shopify** if it does not go by itself).

Can one product have a different price on each website?

Yes. Every website has its own formula, and a rule can give a group of products its own markup.

The vendor's post shows a retail price, not our cost. What then?

The app always treats the number as your cost. Type your real cost in **Cost price (₹)** — what you type is kept.

Will the AI make things up?

It is told to use only the facts given and never to invent a fabric or size. Read the first listings of each website; anything you correct is kept.

What does the AI cost?

One AI call reads each product, and one call writes each listing on each website. Haiku is the cheapest; use Sonnet or Opus only where the writing really matters.

Are prices with or without GST?

The app sends the number from your formula. Whether Shopify treats prices as including tax is set in Shopify (Settings → Taxes and duties).

Who can see the Shopify keys?

Nobody. They are stored on the server, used only to talk to Shopify, and never shown again in the app.

What happens when I pause automation?

Nothing runs on its own until you switch it back on; buttons on a product still work. When it is switched on again it carries on from where it stopped.

Two of us edited the same listing. Whose change stays?

The last one saved. Close and reopen a listing before editing if someone else may have just changed it.

DailyOffice · Playbook — Product Directory — User Guide